

Instruções

Peso: 0,5 ponto | **Entrega:** digital, 07/10 | **Foco:** Funções e modularização

Contexto

Neste checkpoint o quiz ganha um menu interativo e a lógica é dividida em funções. A modularização torna o código mais legível, mais fácil de testar e prepara o sistema para receber arquivos e structs nos próximos CPs.

Descrição

O programa deve oferecer um menu com as opções: jogar, cadastrar perguntas, listar perguntas, exibir estatísticas e sair. Cada opção deve ser implementada em uma função específica. Pelo menos uma função deve retornar um valor e pelo menos uma deve ser void.

Requisitos obrigatórios

- Menu interativo controlado por while e switch.
- Opções: 1. Jogar, 2. Cadastrar, 3. Listar, 4. Estatísticas, 0. Sair.
- Pelo menos uma função com retorno (ex.: `int calcularPontuacao(...)`).
- Pelo menos uma função void (ex.: `void exibirMenu(void)`).
- Funções separadas para cadastrar, listar, jogar e estatísticas.
- Validação de entrada nas opções do menu.

- Estatísticas: total de perguntas cadastradas e pontuação média das partidas.

Exemplo de comportamento esperado

Exemplo de menu:

```
≡≡≡ Quiz ≡≡≡
```

```
1. Jogar
2. Cadastrar pergunta
3. Listar perguntas
4. Estatísticas
0. Sair
Opcao: 1
```

```
Pergunta 1: Quanto e 5 + 3?
```

```
A) 5
B) 8
C) 9
D) 7
```

```
Resposta: B
```

```
Correto!
```

```
...
```

```
Placar: 4/5 (80%)
```

Dica: use protótipos no início do arquivo, definições depois e chame as funções dentro do menu. Funções que não precisam devolver valor devem ser `void`.

Estrutura de pastas esperada

O projeto deve ser organizado nos três arquivos ensinados em aula:

```
meu-projeto-c/
├─ main.c      (menu e função main)
├─ funcoes.c   (definições das funções)
└─ funcoes.h   (protótipos – o contrato)
```

- `main.c` : menu e coordenação do programa.
- `funcoes.c` : implementação das funções.
- `funcoes.h` : protótipos, incluído com `#include "funcoes.h"` .
- `README.md` : instruções de compilação, execução e descrição do projeto.

Compilação: `gcc main.c funcoes.c -o programa`

Restrições

- Ainda sem struct.
- Ainda sem arquivos.

Critérios de avaliação

- Código compila sem erros.
- Uso correto de protótipos, definições e chamadas.
- Menu com while e switch funcional.
- Pelo menos uma função com retorno e uma void.
- Estatísticas calculadas corretamente.

Entrega

Organize o projeto nos três arquivos ensinados em aula: `main.c` (menu e main), `funcoes.c` (definições) e `funcoes.h` (protótipos), mais um `README.md` com instruções. Compacte a pasta do projeto em `.zip` e envie pelo Blackboard.

Checklist de autoavaliação

- ☐ Programa compila sem erros
- ☐ Todos os requisitos obrigatórios estão implementados
- ☐ Casos de teste foram executados antes do envio
- ☐ Código está organizado e comentado
- ☐ Arquivos de entrega estão completos e compactados em .zip

